

MULTI-SOURCE CANDIDATE DATA TRANSFORMER

TECHNICAL DESIGN SPECIFICATIONS

1. PROBLEM STATEMENT

Recruitment platforms collect candidate information from multiple heterogeneous sources, including recruiter exports, Applicant Tracking Systems (ATS), resumes, professional profiles, and recruiter notes. These sources differ in structure, naming conventions, completeness, and data quality. The purpose of this project is to design a deterministic, explainable, and configurable Candidate Data Transformation Pipeline that converts heterogeneous candidate information into a single standardized Master Candidate Record (MCR) without modifying the core processing logic.

2. PIPELINE ARCHITECTURE



3. CANONICAL OUTPUT SCHEMA (MCR)

Field Name	Data Type / Spec Representation
candidate_id	string (Internally generated unique identifier)
full_name	string (Normalized candidate name)
emails[]	array of validated email addresses
phones[]	array of validated phone numbers in E.164 format
location	object (Normalized geographical city, region, country)
skills[]	array of canonical lookup names with confidence
experience[]	array of structured employment history records
education[]	array of structured academic history records
provenance[]	array recording origin and derivation method
overall_confidence	float measuring complete profile reliability

4. DATA NORMALIZATION STRATEGY

- **Full Name:** Trim whitespace, remove duplicate spaces, standardize capitalization.
- **Email:** Convert to lowercase, trim whitespace, validate format, remove duplicates.
- **Phone:** Remove formatting characters and convert to E.164 format where possible.
- **Country:** Convert to ISO 3166 Alpha-2 country codes.
- **Dates:** Convert to YYYY-MM or YYYY-MM-DD depending on available precision.
- **Skills:** Map synonymous skill names to a canonical representation using lookup table.

5. MERGE & CONFLICT RESOLUTION

Match Keys (Priority Order): 1. Exact email → 2. Exact phone → 3. Name + email → 4. Name + phone.

Winner Rules (Per Field):

- Prefer Recruiter CSV over resume-derived values.
- Resume vs. resume conflict → keep the most complete value.
- Still tied → prefer the most recent value, then the longest / most specific.
- Within-source duplicates removed via exact keys before merge.

6. CONFIDENCE SCORING FRAMEWORK

Deterministic score clamped to [0.0, 1.0]; stored as overall_confidence on the record:

- **Base Score (Per Candidate):** 0.50
- **Boosters:** +0.15 Valid email | +0.10 Valid phone | +0.10 Name present | +0.05 Title present | +0.05 ea Skills (≥3) / multi-source.
- **Penalties:** -0.10 ea Missing critical field.

7. RUNTIME OUTPUT CONFIGURATION

Projection config (JSON) controls: Field selection (include/exclude) & renames, Output nesting/flattening rules, and Include/exclude confidence & provenance. Output validation checks required fields present, data types match, and blocks unknown fields (strict mode) before the JSON is written.

8. EDGE CASES & SCOPE BOUNDARIES

- **Missing/Invalid Email:** Use phone or name+phone for identity; mark email null.
- **Corrupted/Unreadable File:** Log error; continue with available sources.
- **Incomplete Recruiter Row:** Skip row or process with available fields.
- **Conflicting Identity Fields:** Keep both in provenance; prefer recruiter; flag conflict.
- **Out of Scope:** Fuzzy/ML entity resolution, real-time streaming, non-PDF/DOCX formats (images, RTF), external enrichment (LinkedIn/GitHub APIs), OCR for scanned resumes, UI/ Dashboard, and DB persistence.